```cpp
// 图形管理类
class ShapeManager {
private:
    vector<Shape*> shapes;  // 存储所有图形的指针
public:
    ~ShapeManager() {
        for (Shape* shape : shapes) {
            delete shape;
        }
    }

    void addShape(Shape* shape) {
        shapes.push_back(shape);
    }

    void removeShape(int index) {
        if (index >= 0 && index < shapes.size()) {
            delete shapes[index];
            shapes.erase(shapes.begin() + index);
        } else {
            cout << "WRONG" << endl;
        }
    }

    void displayShapes() const {
        for (const Shape* shape : shapes) {
            shape->display();
            cout << "面积: " << shape->area() << ", 周长: " << shape->perimeter() << endl;
        }
    }
};
```

编程题＃1： 魔兽世界终极版

【解答】

主要分为四大类：headquarter、warrior、weapon、city；于是定义了四个头文件，分别对这四个大类的函数与属性进行定义。

（1） weapon 类

weapon 类是最简单的类，主要实现两个功能：1.对武器赋予攻击力 2.判断武器是否到达使用限制。是，则 delete，认为武士失去该武器。

对于判断是否到达使用限制，在基类中设置虚函数，对对应的武器种类进行具体的重写。

头文件内容（截取）：

```cpp
#pragma once
#ifndef WEAPON_H
#define WEAPON_H
#include<iostream>
class weapon {

public:
    double power;
    std::string kind;
    void operator=(const weapon& w);
    virtual void print_info()=0;
    virtual bool lost_weapon() {//如果为true，则视为武士不再拥有武器
        return true;
    };
    virtual ~weapon() {
        kind.clear();
        power = 0;
    }
};
class sword : public weapon {
private:
public:
    sword() {
        power = 0;
        kind.assign("sword");
    }
    ~sword() { }
    bool lost_weapon() override{//在每次使用武器之后，调用这个函数判断武器是否仍然存在
        power = int(power * 0.8);
        if (power <= 0) {
            return true;
        }
        return false;
    }
    void print_info() {
```

（2） warrior 类

warrior 类主要实现的操作可以分为三方面：

1.初始：对武士武器的分配，不同种类武士具有不同特性的实现。

2.前进：前进相关的操作，让武士与其所在城市、headquarters 胜利条件相挂钩。

3.战斗：进行战斗，产生伤害或受到伤害，win or lose

这里对如何实现不一一赘述，挑几个例子展示：

1.初始：如，对 iceman 前进时，hp 减少伤害增加；如判断狮子是否会逃跑。

```
281
282 void iceman::decrease_life() {//在前进后调用
283     if (step % 2 == 0) {
284         if (my_hp > 9) {
285             my_hp-= 9;
286             power += 20;
287         }
288         else {
289             HP = 1;
290             power += 20;
291         }
292     }
293     else {
294         return;
295     }
296
297 }
298
299 void lion::set_loyality(int i) {
300     loyality = i;
301 }
302
303 void lion::decrease_loyality() {
304     loyality -= K;
305     if (loyality <= 0) {
306         to_run = true;
307     }
308     return;
309 }
```

2．前进：

为了更方便的判断是否到达目的地/所在城市，直接在 warrior 基类中添加属性

int In_city;//所在的城市编号

int destination;//目的地

```
prepare_war                    Warrior                    fight_back()
40  bool Warrior::step_on( ) {//0为红方基地，n+1为蓝方基地，1-n为城市
41      //对雪人进行特殊处理
42      if (get_kind() == "iceman") {
43          iceman* i = dynamic_cast<iceman*> (this);
44          i->step++;
45          i->decrease_life();
46      }
47      if (belong_headquater == "RED"&&In_city<destination) {
48          In_city++;//
49      }
50      else if (belong_headquater == "RED" && In_city == destination) {
51          In_city++;
52          print_name();
53          arrive_destination = true;
54          std::cout<<"reached BLUE headquarter with "<<my_hp<<" elements and force "<<power<<std::endl;
55          return true;
56      }
57      if(belong_headquater=="BLUE"&&In_city>1) {
58          In_city--;
59      }
60      else if (belong_headquater == "BLUE" && In_city == 1) {
61          In_city--;
62          print_name();
63          arrive_destination = true;
64          std::cout << "reached RED headquarter with " << my_hp << " elements and force " << power << std::endl;
65          return true;
66      }
67
68      print_name();
69      std::cout << "marched to city " << In_city <<" with " << my_hp << " elements and force " << power << std::endl;
70      return false;
71  }
```

3.战争：

在这里，我将放箭和使用炸弹划为城市的行为，于是，对 warrior 类而言，只需要考虑主动进攻/反击/战死

对于主动进攻/反击，我在基类函数中完成了虚函数定义，在两个特例（ninja 和 wolf）进行了 override。因为这两种武士具有多个武器，不能像基类那样一以盖之。

进攻/反击函数，返回值为所产生的伤害；get_hurt(double hurt)函数，能让武士受到 hurt 值得伤害，也为总体的战争实现基础。

（warrior 类主动进攻）

```
double Warrior::start_war() {//进行主动攻击，返回值为造成的伤害

    double hurt = power;
    bool is_lost=false;
    if (my_weapon == nullptr) {
        return hurt;
    }
    if (my_weapon->kind == "sword") {
        hurt += my_weapon->power;
        sword* s = dynamic_cast<sword*>(my_weapon);
        is_lost = s->lost_weapon();
    }
    //std::cout << "[DEBUG]主动攻击测试"<<hurt << std::endl;
    //std::cout << "[DEBUG]武器类型" << my_weapon.kind <<my_weapo
    if (is_lost) {
        delete my_weapon;
        my_weapon = nullptr;
    }
    return hurt;
}
```

（wolf 类主动进攻）

```cpp
double wolf::start_war() {
    double hurt = power;
    bool is_lost=false;
    if (my_sword != nullptr) {
        hurt += my_sword->power;
        is_lost = my_sword->lost_weapon();
    }
    if (is_lost) {
        delete my_sword;
        my_sword = nullptr;
    }
    return hurt;
}
```

（3） city 类

定义了 city 类和 cities 类，后者是 city 的 vector 数组。

主要实现三个功能：1.发生射箭/使用炸弹 2. 发生战争 3.生命元被夺取

1.判断箭/炸弹

在 city 类中，设置有 warrior* 指针和布尔值 have_arrow/have_bomb，当其得到赋值时，对武士进行分析，如果有箭/炸弹，再进行进一步分析。是否达到使用条件。

分析过程较冗杂，不一一分析

（截取部分）

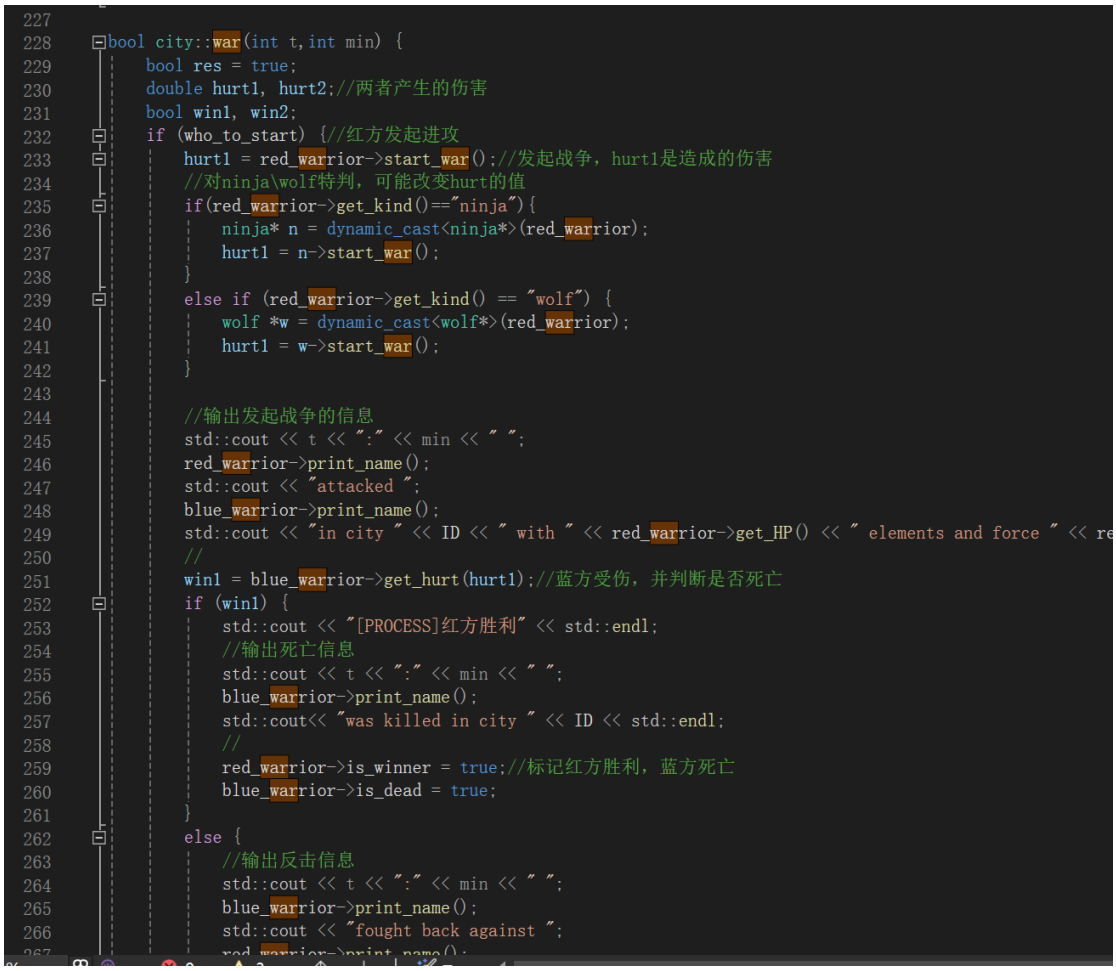
```cpp
int city::predict_bomb() {//*************需要完善*************
    //0:不用，-1：红方用，1：蓝方用
    int use_bomb = 0;
    if (!red_have_bomb&&!blue_have_bomb) {
        return 0;
    }
    if (!red_warrior || !blue_warrior) {
        return 0;
    }
    double hurt1,hurt2;
    if (red_have_bomb && who_to_start) {//红方拥有炸弹&&发起进攻
        hurt1 = red_warrior->start_war();
        //**********加入特判
        if (red_warrior->get_kind() == "ninja") {
            ninja *n = dynamic_cast<ninja*>(red_warrior);
            hurt1 = n->start_war();
        }
        else if (red_warrior->get_kind() == "wolf") {
            wolf* w = dynamic_cast<wolf*>(red_warrior);
            hurt1 = w->start_war();
        }
        //********特判结束

        if (blue_warrior->get_HP() > hurt1) {//蓝方不死，发起反击
            hurt2 = blue_warrior->fight_back();
        //特判
            if (blue_warrior->get_kind() == "ninja") {
                ninja *n2 = dynamic_cast<ninja*>(blue_warrior);
                hurt2 = n2->fight_back();
            }
            else if(blue_warrior->get_kind()=="wolf"){
                wolf* w2 = dynamic_cast<wolf*>(blue_warrior);
                hurt2 = w2->fight_back();
            }
            //*******特判结束
            if (red_warrior->get_HP() <= hurt2) {//红方受反击而死
                use_bomb = -1;
            }
        }
    }//红方拥有炸弹&&发起进攻
```

2.发生战争

主要调用 warrior 里封装好的函数，实现一个交互

(截取部分代码)



```
227
228 bool city::war(int t,int min) {
229     bool res = true;
230     double hurt1, hurt2;//两者产生的伤害
231     bool win1, win2;
232     if (who_to_start) {//红方发起进攻
233         hurt1 = red_warrior->start_war();//发起战争，hurt1是造成的伤害
234         //对ninja\wolf特判，可能改变hurt的值
235         if(red_warrior->get_kind()=="ninja"){
236             ninja* n = dynamic_cast<ninja*>(red_warrior);
237             hurt1 = n->start_war();
238         }
239         else if (red_warrior->get_kind() == "wolf") {
240             wolf *w = dynamic_cast<wolf*>(red_warrior);
241             hurt1 = w->start_war();
242         }
243
244         //输出发起战争的信息
245         std::cout << t << ":" << min << " ";
246         red_warrior->print_name();
247         std::cout << "attacked ";
248         blue_warrior->print_name();
249         std::cout << "in city " << ID << " with " << red_warrior->get_HP() << " elements and force " << re
250         //
251         win1 = blue_warrior->get_hurt(hurt1);//蓝方受伤，并判断是否死亡
252         if (win1) {
253             std::cout << "[PROCESS]红方胜利" << std::endl;
254             //输出死亡信息
255             std::cout << t << ":" << min << " ";
256             blue_warrior->print_name();
257             std::cout<< "was killed in city " << ID << std::endl;
258             //
259             red_warrior->is_winner = true;//标记红方胜利，蓝方死亡
260             blue_warrior->is_dead = true;
261         }
262         else {
263             //输出反击信息
264             std::cout << t << ":" << min << " ";
265             blue_warrior->print_name();
266             std::cout << "fought back against ";
```

3.夺取生命元

主要分为两个可能：1.只有一个武士 2.战斗后夺取

（只有一个武士）

```cpp
void cities::OneWarrior_took_hp(HeadQuarter* red, HeadQuarter* blue,int t,int min) {
    for (int i = 1; i < city_list.size(); i++) {
        city *c = city_list[i];
        if (c->red_warrior && c->blue_warrior) {
            continue;
        }
        if (!c->red_warrior && !c->blue_warrior) {
            continue;
        }
        int gain_hp;
        if (c->red_warrior && !c->blue_warrior) {
            gain_hp = c->took_HP();
            std::cout << t << ":" << min << " ";
            c->red_warrior->print_name();
            std::cout<<"earned "<<gain_hp<<" elements for his headquarter"<<std::endl;
            red->add_HP(gain_hp);
        }
        if (!c->red_warrior && c->blue_warrior) {
            gain_hp = c->took_HP();
            std::cout << t << ":" << min << " ";
            c->blue_warrior->print_name();
            std::cout << "earned " << gain_hp << " elements for his headquarter" << std::endl;
            blue->add_HP(gain_hp);
        }

    }
    return;
}

```

（两个武士发生战争，后夺取生命元）

```cpp
void cities::AfterWar_took_hp(HeadQuarter* red, HeadQuarter* blue,int t,int min) {
    for (int i = 1; i < city_list.size(); i++) {
        city* c = city_list[i];
        if (!c->have_war) {//没有发生战争
            continue;
        }
        int gain_hp;
        if (c->red_warrior->is_winner) {
            gain_hp = c->took_HP();
            std::cout << t << ":" << min << " ";
            c->red_warrior->print_name();
            std::cout << "earned " << gain_hp << " elements for his headquarter" << std::endl;
            red->add_HP(gain_hp);
            c->red_warrior->is_winner = false;//重置武士信息
        }
        else if (c->blue_warrior->is_winner) {
            gain_hp = c->took_HP();
            std::cout << t << ":" << min << " ";
            c->blue_warrior->print_name();
            std::cout << "earned " << gain_hp << " elements for his headquarter" << std::endl;
            blue->add_HP(gain_hp);
            c->blue_warrior->is_winner = false;//重置武士信息
        }
        else {
            continue;
        }
    }
}
```

（4） headquarter 类

主要实现：生产武士/奖励武士/清除死亡武士/汇报状态

依次简单封装即可。

类定义：

```
class HeadQuarter {
private:
    std::string name;
    int HP_sum;
    int make_warrior_order[5];//0:dragon,1:ninja,...
    int sum_cnt_warriors = 0;
    int cnt_of_different_warriors[5] = { 0 };//不妨设顺序为dragon、ninja、iceman、lion、
    std::vector<Warrior*> list_of_warriors;
    bool is_stop_creating = false;
    int stop_creating_at=0;
public:
    int des;//目的地，红方为n+1，蓝方为0
    int base;//基地位置，红方为0，蓝方为n+1
    int TakeDown;//进入对方基地的武士个数
    HeadQuarter(int m, std::string new_name);
    ~HeadQuarter();
    void show_kind_of_warrior(int order);

    void set_order_for_HeadQuarter(int n);

    int follow_order_to_create(int order);

    void create_warrior(int t);

    bool get_is_stop_creating();

    int get_HPSUM();
    std::vector<Warrior*> get_list_of_warriors();
    Warrior* get_warrior();
    void add_HP(int hp);
    //按照时间点的一系列操作
    void lion_run(int t,int min);//狮子逃跑
    void march(int t,int min);//武士前进

    void reward();//战斗之后发放奖励（对赢了的）
    void clear_dead();//清除死亡的武士
    void report_hp();//汇报生命元
    void report_warrior(int t,int min);//让武士汇报武器情况
};
```

生产武士：沿用之前作业的代码。
奖励武士：赢了战争的武士用 is_winner 标记，方便进行奖励

```
void HeadQuarter::reward() {//由近至远奖励，应该倒序遍历vector数组
    for (int i = list_of_warriors.size()-1; i>=0; i--) {
        if (list_of_warriors[i]->is_winner == true) {
            if (HP_sum >= 8) {
                list_of_warriors[i]->add_HP(8);
                HP_sum -= 8;
            }
            else {//剩余HP不足奖励，退出
                return;
            }
        }
    }
};
```

清除死亡武士：如果死亡，将武士从 vector 数组中清除。

```
276
277     void HeadQuarter::clear_dead() {
278         for (int i = 0; i < list_of_warriors.size(); i++) {
279             if (list_of_warriors[i]->is_dead == true) {
280                 list_of_warriors.erase(list_of_warriors.begin() + i);
281                 i--;
282             }
283         }
284     };
```

(字段) std::vector<Warrior *> HeadQuarte

联机搜索

汇报状态：不详细解释。

（5） 最后实现 Round 函数，代表着每一个整点发生的事件顺序。（其中创造武士放在了调用 Round 函数前）在 round 函数中，完成武士前进一步后，判断司令部是否被攻陷，如果是，则终止函数并返回。

```
2   //t代表整点
3   int Round(HeadQuarter *red,HeadQuarter *blue,cities *c,int t) {
4       int min = 0;
5       //0分，创造武士******************不完善*****************
6       //red->create_warrior(t);
7       //blue->create_warrior(t);
8       //5分，狮子逃跑
9       min = 5;
10      red->lion_run(t,min);
11      blue->lion_run(t,min);
12      //10分，武士前进
13      min = 10;
14      red->march(t,min);
15      blue->march(t,min);
16      if (red->TakeDown >= 2) {
17          return -1;
18      }
19      if (blue->TakeDown >= 2) {
20          return 1;
21      }
22      c->warrior_enter_city(red, blue);//对城市而言，使其对武士指针，指向进入的武士
23      //20分，生成HP
24      c->cities_create_hp();
25      //30分，只有一个武士的城市失去生命元
26      min = 30;
27      c->OneWarrior_took_hp(red, blue,t,min);
28      //35分，放箭
29      min = 35;
30      c->use_arrow();//从城市群对放箭分析.对城市删除了武士指针（指向nullptr）
31      //red->clear_dead();//清理死亡武士
32      //blue->clear_dead();
33      //38分，评估炸弹的使用
34      min = 38;
35      c->use_bomb(t,min);
36      //40分，战争
37      min = 40;
38      c->all_war(t, min);//包含进攻/反击/战死/欢呼/升旗
39      red->clear_dead();//清理死亡武士，并对胜者奖励
40      blue->clear_dead();
41      red->reward();
42      blue->reward();
43      c->AfterWar_took_hp(red, blue,t,min);//指挥部收取生命元
```

再在 main 函数中集中调用。

（6） main 函数

```cpp
int wolf::w_power = 0;
double lion::K = 0;
int main() {
    std::string r = "RED";
    std::string b = "BLUE";
    int cnt;
    std::cin >> cnt;
    int M,N,r_arrow,K,T;
    for (int i = 0; i < cnt; i++) {
        std::cin >> M>>N>>r_arrow>>lion::K>>T;
        HeadQuarter R(M, r);
        HeadQuarter B(M, b);
        cities C(N);
        C.R = r_arrow;
        R.des = N + 1, B.des = 0;
        R.base = 0,B.base = N + 1;
        std::cin >> dragon::HP>>ninja::HP>>iceman::HP>>lion::HP>>wolf::HP;
        std::cin >> dragon::d_power >> ninja::n_power >> iceman::i_power >> lion::l_power >> wolf::w_power;
        R.set_order_for_HeadQuarter(0);//0\1是固定模式，也可以自己输入武士顺序；
        B.set_order_for_HeadQuarter(1);
        int t = 0;
        int res;

        std::cout<<"Case:"<<i+1<<std::endl;
        while (t<T) {
            R.create_warrior(t);
            B.create_warrior(t);
            res = Round(&R, &B, &C, t);
            if (res == 0) {
                t++;
                continue;
            }
            if (res == 1) {
                std::cout << t << ":10 ";
                std::cout<<"red headquarter was taken"<<std::endl;
                break;
            }
            else if (res == -1) {
                std::cout << t << ":10 ";
                std::cout<<"blue headquarter was taken"<<std::endl;
                break;
            }
```

样例输入

```
1

20 1 10 10 1000

20 20 30 10 20

5 5 5 5 5
```

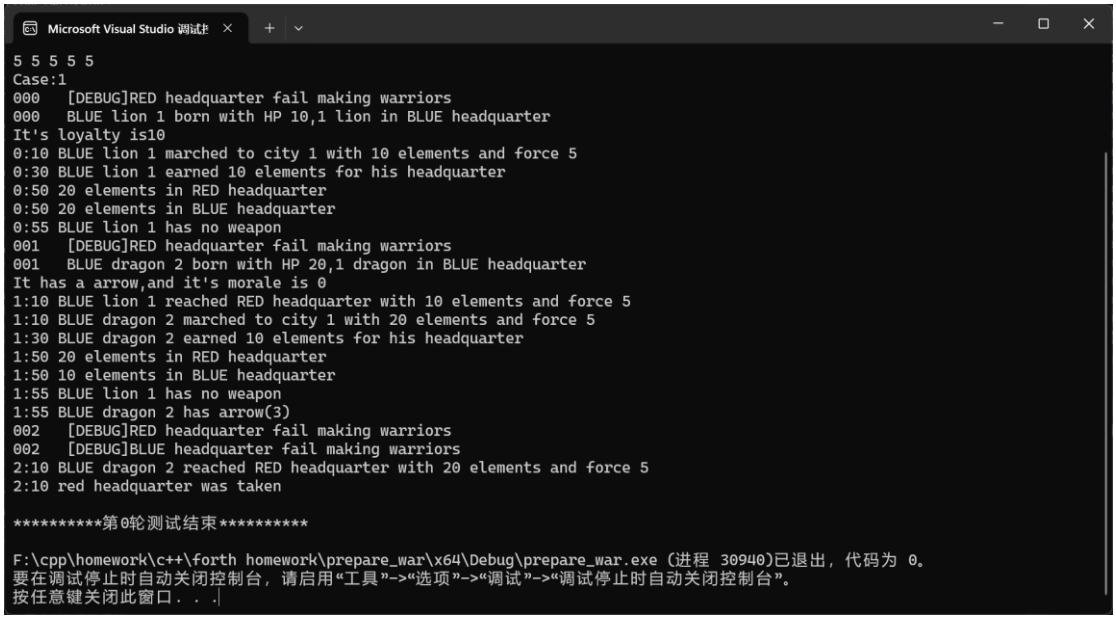
样例输出

```
Case 1:
000:00 blue lion 1 born
Its loyalty is 10
000:10 blue lion 1 marched to city 1 with 10 elements and force 5
000:30 blue lion 1 earned 10 elements for his headquarter
```

```
000:50 20 elements in red headquarter
000:50 20 elements in blue headquarter
000:55 blue lion 1 has no weapon
001:00 blue dragon 2 born
Its morale is 0.00001:10 blue lion 1 reached red headquarter with 10 elemen
ts and force 5
001:10 blue dragon 2 marched to city 1 with 20 elements and force 5
001:30 blue dragon 2 earned 10 elements for his headquarter
001:50 20 elements in red headquarter
001:50 10 elements in blue headquarter
001:55 blue lion 1 has no weapon
001:55 blue dragon 2 has arrow(3)
002:10 blue dragon 2 reached red headquarter with 20 elements and force 5
002:10 red headquarter was taken
```

【测试结果】

1.按照样例输入，得到的输出



2.输入自己设计的测试样例

输入：1

50 5 5 10 20

10 10 20 10 30

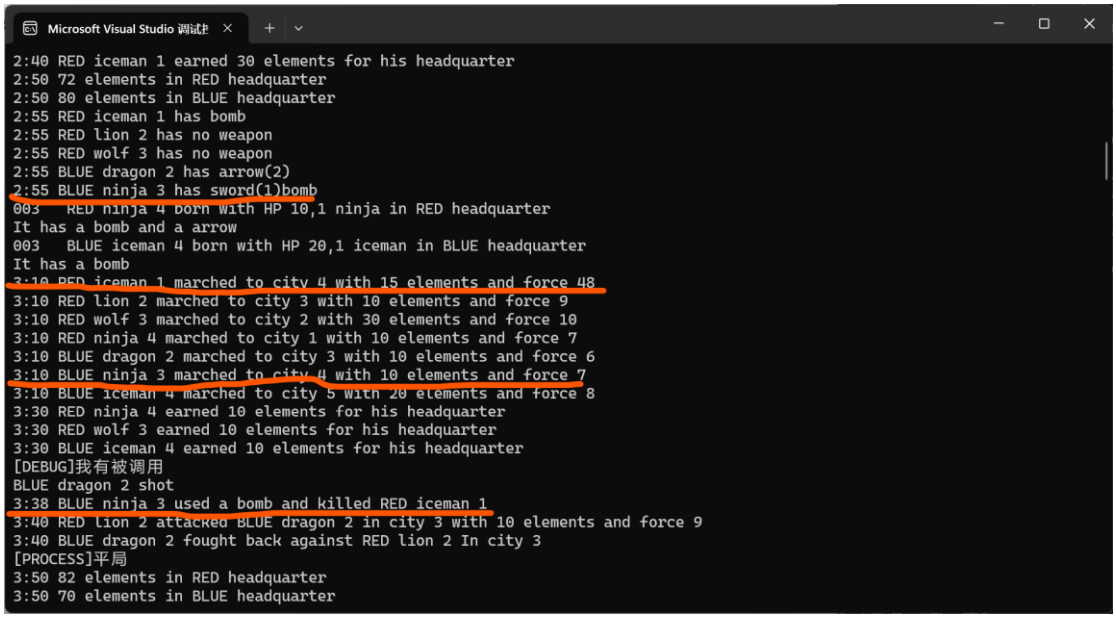
6 7 8 9 10

得到结果：

1.成功完成是否使用炸弹的判定，以及成功使用炸弹

（根据标注的信息分析，red iceman 与持有炸弹的 ninja 在 city 4 相遇，这时 iceman 伤害值超过 ninja 生命值，ninja 必死，所以使用炸弹）

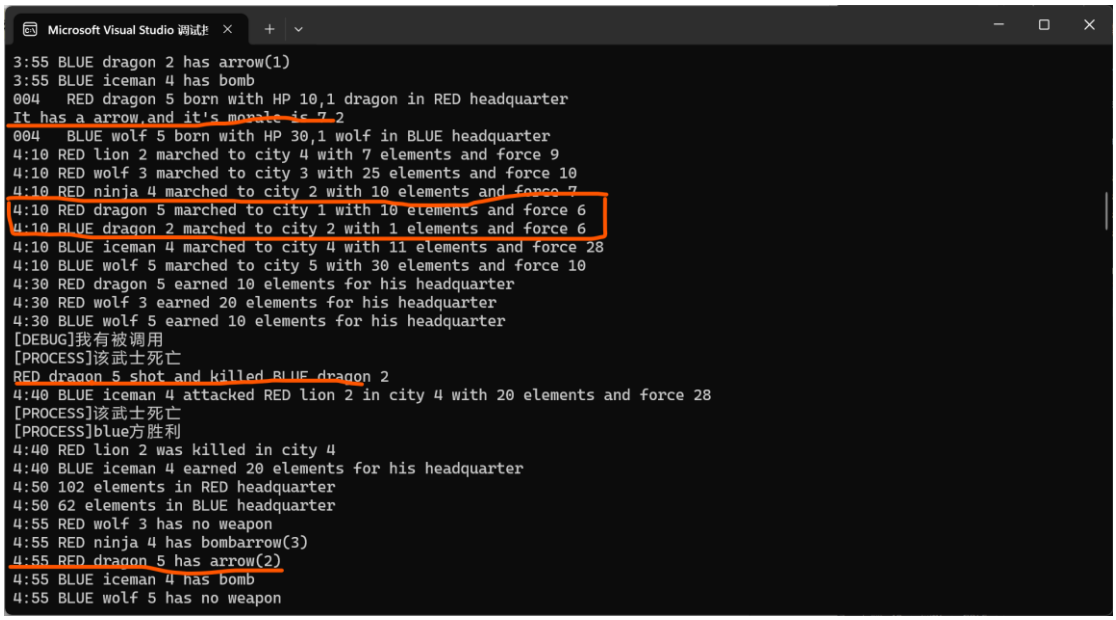
（如下图）



2.成功放箭杀死敌人，并正确改变箭的剩余次数

（分析：此时 city 1 的 red dragon 有剩余次数 3 的箭，city 2 存在 blue ninja，dragon 放箭射杀，在汇报武器情况时，剩余次数改变）

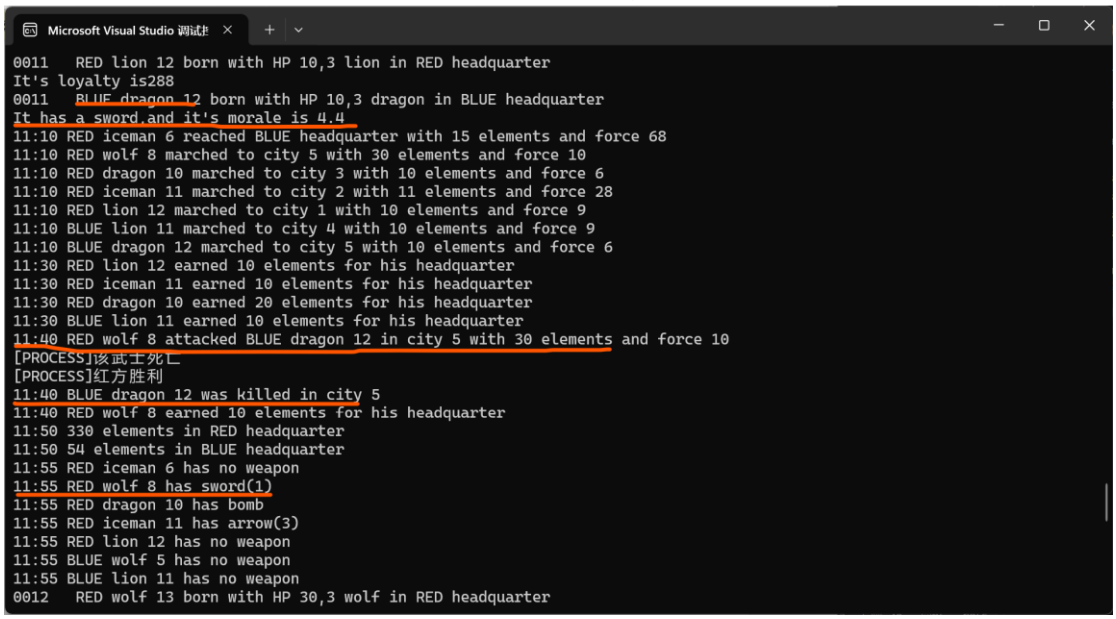
（如下图）



3.wolf 杀死敌人并成功捡起武器

（如图，red wolf 8 杀死了 blue dragon 12，dragon 12 持有一把 sword，在汇报武器情况时，可以看到 wolf 成功拥有了 sword）

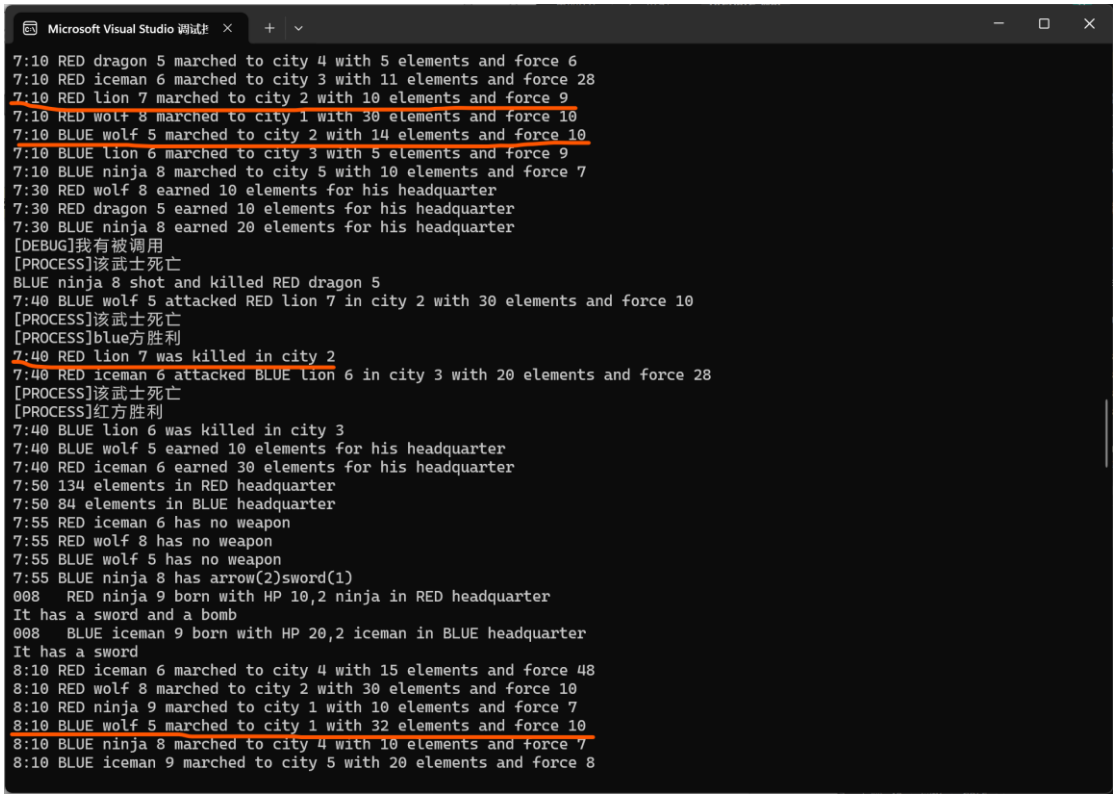
（如下图）



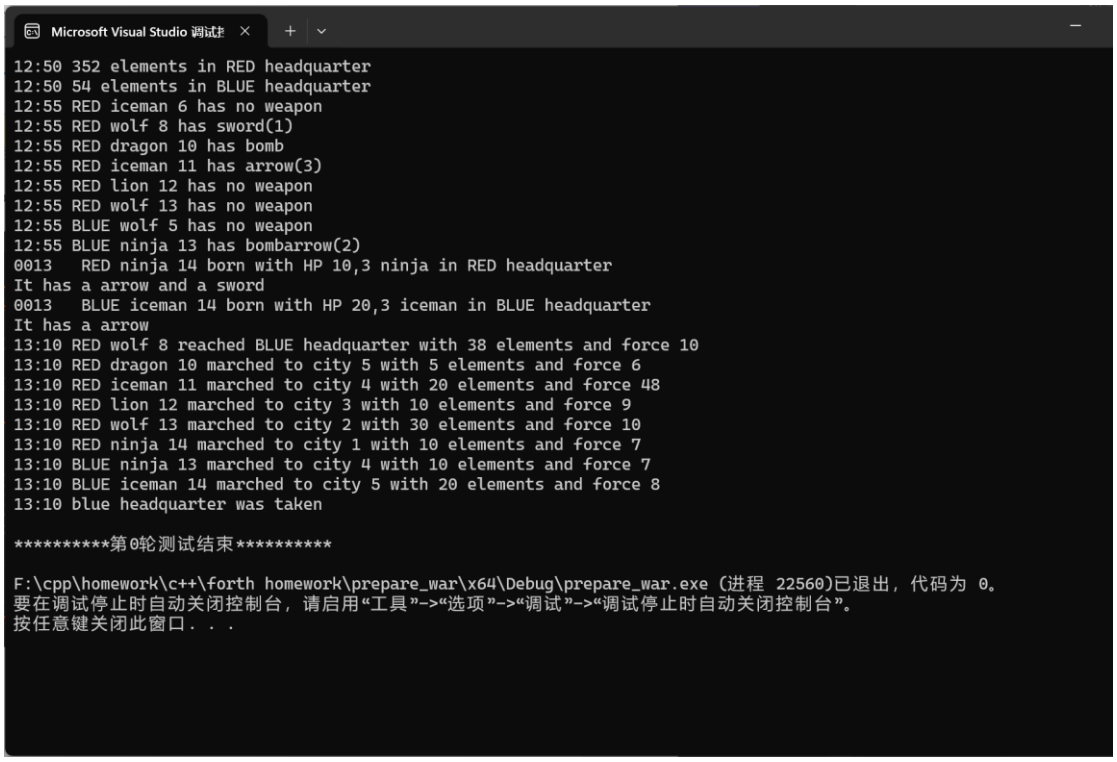
5. lion 死亡并转移生命值

（分析：生命值为 10 的 red lion 7 被生命值为 14 的 blue wolf 5 杀死，blue wolf 5 的生命值：14+10（夺取的生命）+8（奖赏）=32.前进时输出信息，wolf 生命值确实为 32）

如图



6. 测试结束，蓝方被占领


Microsoft Visual Studio 调试控
12:50 352 elements in RED headquarter
12:50 54 elements in BLUE headquarter
12:55 RED iceman 6 has no weapon
12:55 RED wolf 8 has sword(1)
12:55 RED dragon 10 has bomb
12:55 RED iceman 11 has arrow(3)
12:55 RED lion 12 has no weapon
12:55 RED wolf 13 has no weapon
12:55 BLUE wolf 5 has no weapon
12:55 BLUE ninja 13 has bombarrow(2)
0013 RED ninja 14 born with HP 10,3 ninja in RED headquarter
It has a arrow and a sword
0013 BLUE iceman 14 born with HP 20,3 iceman in BLUE headquarter
It has a arrow
13:10 RED wolf 8 reached BLUE headquarter with 38 elements and force 10
13:10 RED dragon 10 marched to city 5 with 5 elements and force 6
13:10 RED iceman 11 marched to city 4 with 20 elements and force 48
13:10 RED lion 12 marched to city 3 with 10 elements and force 9
13:10 RED wolf 13 marched to city 2 with 30 elements and force 10
13:10 RED ninja 14 marched to city 1 with 10 elements and force 7
13:10 BLUE ninja 13 marched to city 4 with 10 elements and force 7
13:10 BLUE iceman 14 marched to city 5 with 20 elements and force 8
13:10 blue headquarter was taken
**********第0轮测试结束**********
F:\cpp\homework\c++\forth homework\prepare_war\x64\Debug\prepare_war.exe (进程 22560)已退出，代码为 0。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口. . .